

DEBUGGING

EXCEPTIONS IN JAVA

- Als Exception bezeichnet man im Allgemeinen:
 - 👉 ein Ereignis, das den normalen Ablauf eines Programms unterbricht
 - 👉 ein Objekt, das Informationen über dieses Ereignis enthält
- Tritt eine Exception auf, so ist es dem Programm überlassen, ...
 - ✅ die Exception zu behandeln
 - ➡ die Exception an den aufrufenden Code weiterzugeben
 - ❌ das Programm zu beenden
- In Java sind Exceptions im Gegensatz zu Errors in der Regel beherrschbar bzw. behandelbar

EXCEPTIONS IN JAVA

CHECKED EXCEPTIONS

- werden während der Kompilierzeit überprüft
- Der Compiler erzwingt, dass diese Exceptions entweder behandelt oder weitergegeben werden
- Beispiele für Checked Exceptions:
 - 👉 `IOException`
 - 👉 `SQLException`
 - 👉 `ClassNotFoundException`

UNCHECKED EXCEPTIONS

- treten zur Laufzeit auf und werden nicht vom Compiler überprüft
- Diese Exceptions können unbehandelt bleiben, ohne dass der Compiler einen Fehler meldet
- Beispiele für Unchecked Exceptions:
 - 👉 `NullPointerException`
 - 👉 `ArrayIndexOutOfBoundsException`
 - 👉 `IllegalArgumentException`

EXCEPTIONS WERFEN

Exceptions können mit dem Schlüsselwort `throw` geworfen werden:



```
1 public void doSomethingDangerous(Object someObject) throws  
   IllegalArgumentException {  
2     if (someObject.getValue() == null) {  
3         throw new IllegalArgumentException("Wert fehlt!");  
4     }  
5     // do something dangerous here safely  
6 }
```



EXCEPTIONS BEHANDELN

Exceptions können mit dem Schlüsselwort try-catch-finally behandelt werden:



```
1  try {
2      // ⚠ Methode, die eine Exception werfen könnte
3      doSomethingDangerous(someObject);
4  } catch (IllegalArgumentException e) {
5      System.out.println("Falsches Argument: " + e.getMessage());
6      // ➡ Behandlung der IllegalArgumentException
7  } catch (Exception e) {
8      // ➡ Behandlung aller anderen Exceptions
9      System.out.println("Sonstiger Fehler: " + e.getMessage());
10 } finally {
11     // ➡ wird immer ausgeführt, egal ob eine Exception aufgetreten ist oder
    nicht
12     System.out.println("Aufräumarbeiten...");
13 }
```

CHECKED EXCEPTIONS BEHANDELN

Checked Exceptions müssen entweder behandelt (try-catch-block) oder weitergegeben werden:

```
1 public String readLineFromFile(String filePath) throws IOException {
2     BufferedReader reader = new BufferedReader(new FileReader(filePath));
3     String line = reader.readLine(); // ⚠️ IOException könnte hier auftreten
4     reader.close();
5     return line;
6 }
```

CUSTOM EXCEPTIONS

Eigene Exceptions können erstellt werden, um spezifische Fehlersituationen zu modellieren:

```
public class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

EXCEPTIONS BEST PRACTICES

✓ DO'S

- **Fail Early:** Exceptions sofort werfen, wenn ein Problem erkannt wird
- **Spezifische Exceptions:** Konkrete Exception-Typen verwenden
- **Aussagekräftige Nachrichten:** Detaillierte Fehlerbeschreibungen
- **Exceptions dokumentieren:** Javadoc für geworfene Exceptions
- **Finally-Block:** Für Ressourcen-Cleanup verwenden

✗ DON'TS

- **Don't Fail Silently:** Niemals Exceptions "verschlucken"
- **Keine Exception für Kontrollfluss:** Exceptions sind keine if-Statements
- **Catch-All vermeiden:** Nicht alle Exceptions pauschal fangen
- **Keine leeren Catch-Blöcke:** Immer angemessen reagieren
- **Stack Trace nicht verlieren:** Original Exception weitergeben

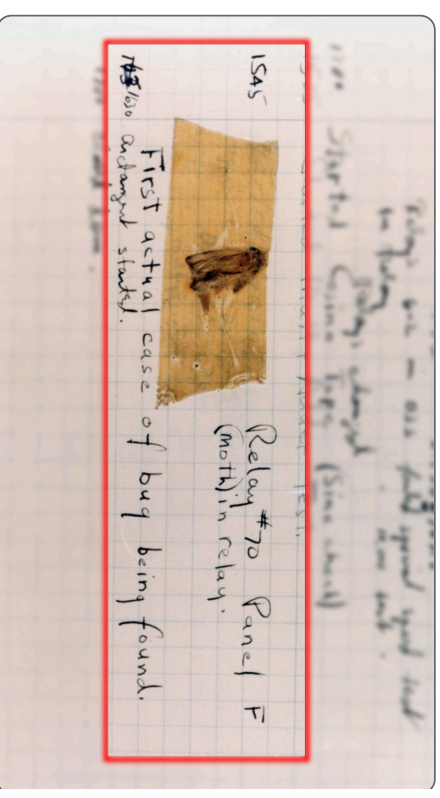
Weiterführende Informationen:

- [OWASP \(Open Worldwide Application Security Project\) Error Handling Guide](#)
- [Blog Post: Mastering Error Handling: A Comprehensive Guide](#)
- [Blog Post: Best Practices of Error Handling in Software Development](#)

DEBUGGING

WAS IST DEBUGGING?

- **Definition:** Der Prozess des Auffindens und Behebens von Fehlern (Bugs) in Software
- **Etymologie:** Begriff geht zurück auf Grace Hopper (1947)
- **Ziel:** Fehlerhafte Programme zum korrekten Verhalten bringen
- **Systematischer Ansatz:** Fehler reproduzieren, lokalisieren und beheben



Der erste "Bug" - 1947

DEBUGGER

SINN UND ZWECK EINES DEBUGGERS

- Fehlerarten in Programmen:
 -  Falsches Ergebnis / unerwartetes Verhalten
 -  Programmabsturz durch Exceptions
 -  Endlosschleifen
 -  Unbehandelte Benutzereingaben
- Debugger-Funktionen:
 -  Schrittweise Code-Analyse (Step Into/Over/Out)
 -  Inspektion von Variableninhalten zur Laufzeit
 -  Breakpoints für gezielte Programmunterbrechung
 -  Analyse der Aufrufhierarchie (Call Stack)
 -  Watch-Expressions für kontinuierliche Überwachung
- Qualitätsmaß: "Defect Density" (Bugs pro KLOC - 1000 Lines of Code)

LOGGING VS DEBUGGING



LOGGING VS DEBUGGING



LOGGING



Produktive Systeme: Überwachung im laufenden Betrieb



Fehleranalyse: Nach dem Auftreten von Problemen



Audit-Trail: Wer hat wann was gemacht?



Alerting: Automatische

Benachrichtigungen bei kritischen

Fehlern



Persistierung: Logs werden (dauerhaft) gespeichert



DEBUGGING



Entwicklungszeit: Aktive Fehlersuche beim Programmieren



Gezieltes Untersuchen: Spezifische Bugs reproduzieren und beheben



Live-Inspektion: Variablen und Zustand in Echtzeit prüfen



Programmfluss verstehen: Schritt-für-Schritt Code-Durchlauf



Hypothesen testen: "Was passiert, wenn...?"

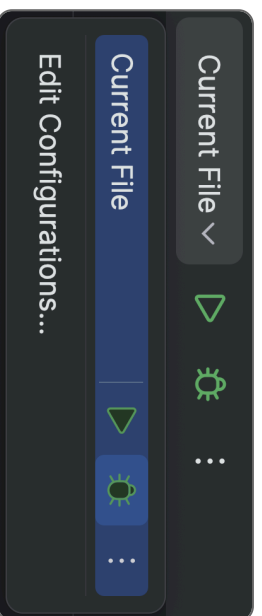


Wichtiger Tipp: Verwendet die Debugging-Tools eurer IDE statt `System.out.println()` oder `console.log()` für die Fehlersuche!

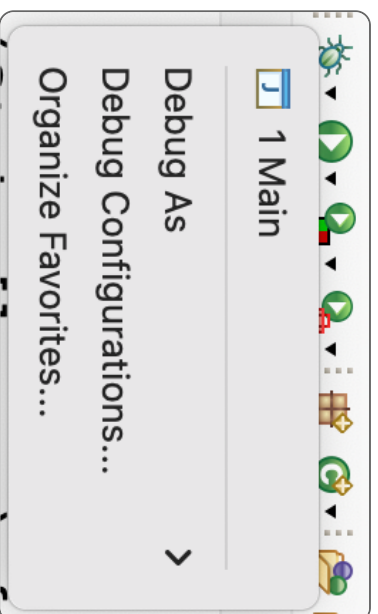
DEBUGGER IN DER IDE

Starten des Debuggers in den verschiedenen IDEs:

INTELLIJ IDEA



ECLIPSE IDE

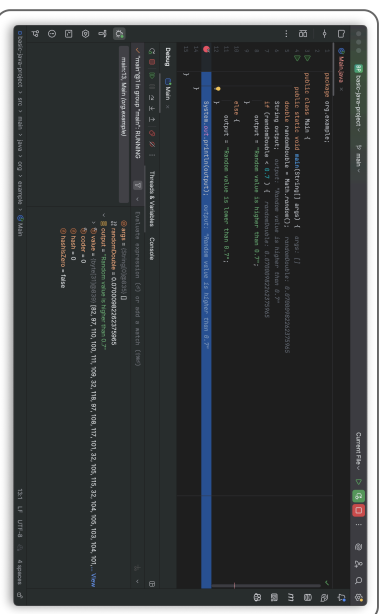


VISUAL STUDIO CODE

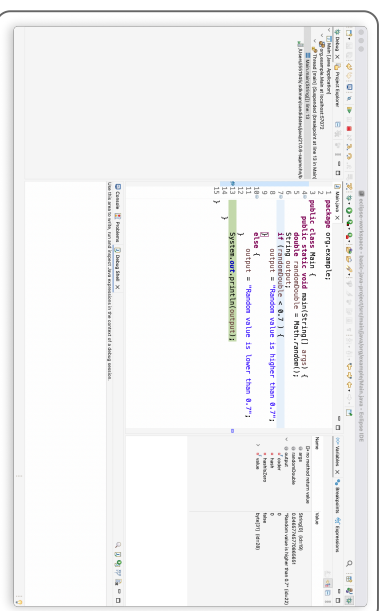


DEBUGGER IN DER IDE

Die Debug-Ansicht in den verschiedenen IDEs:
INTELLIJ IDEA



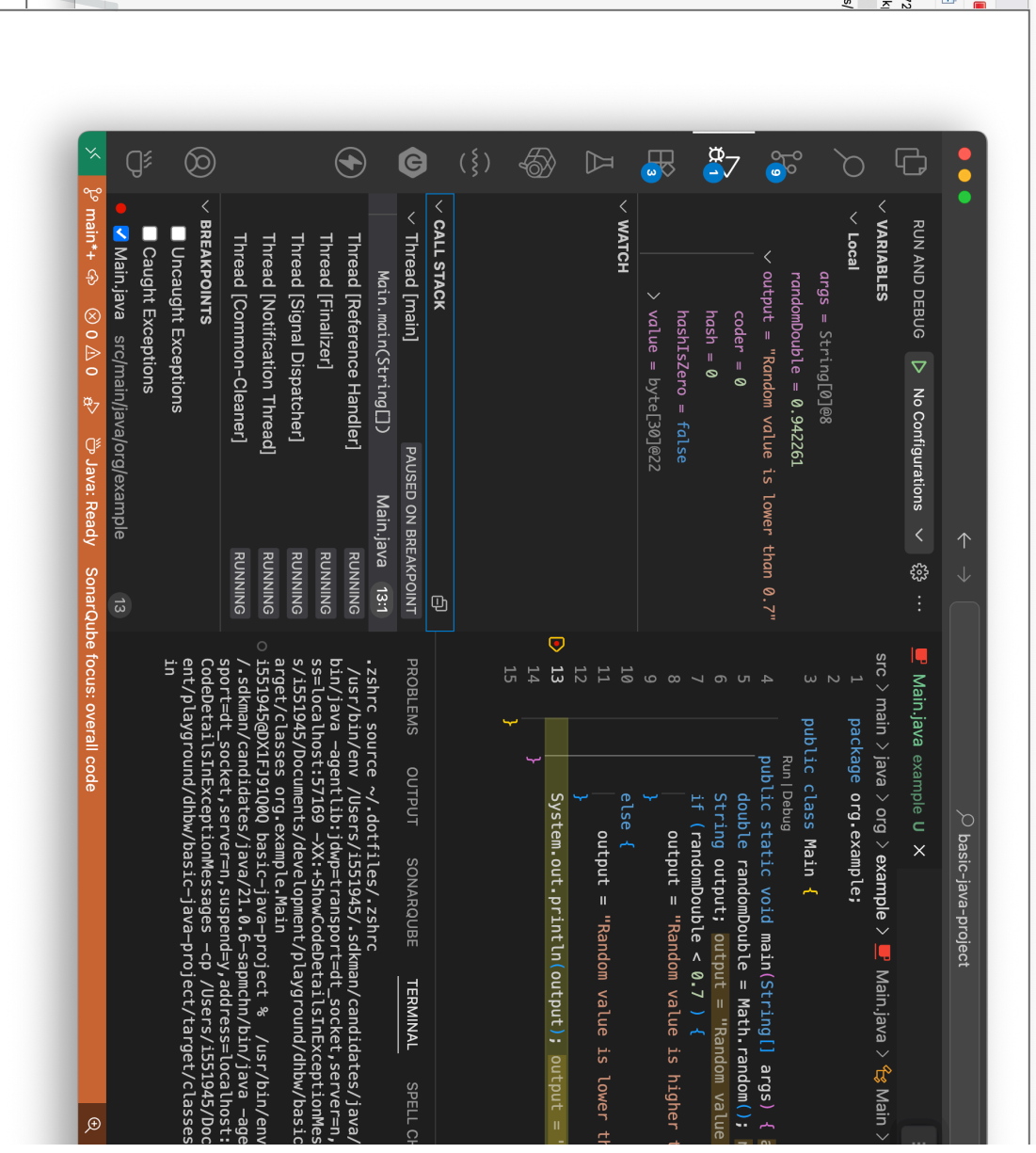
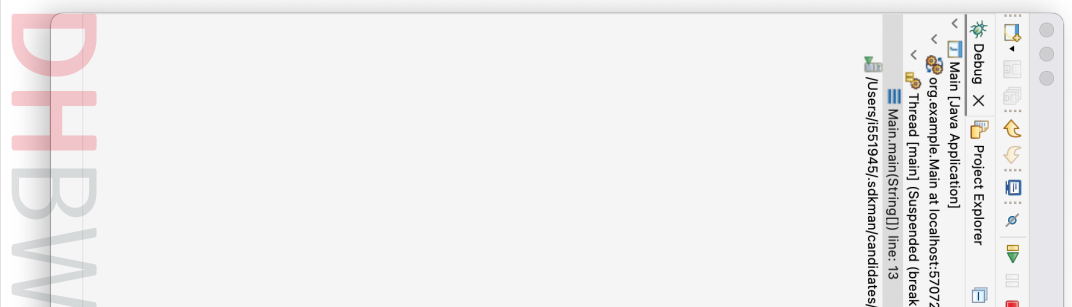
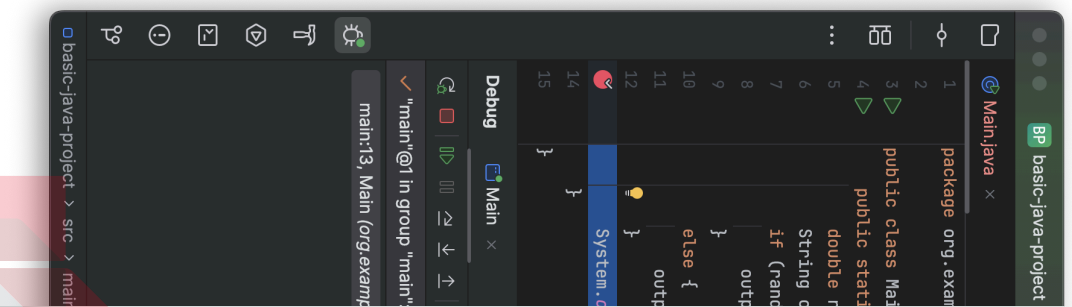
ECLIPSE IDE



VISUAL STUDIO CODE



DEBUGGER IN DER IDE



DEBUGGER IN DER IDE

Step Controls in den verschiedenen IDEs:

INTELLIJ IDEA



ECLIPSE IDE



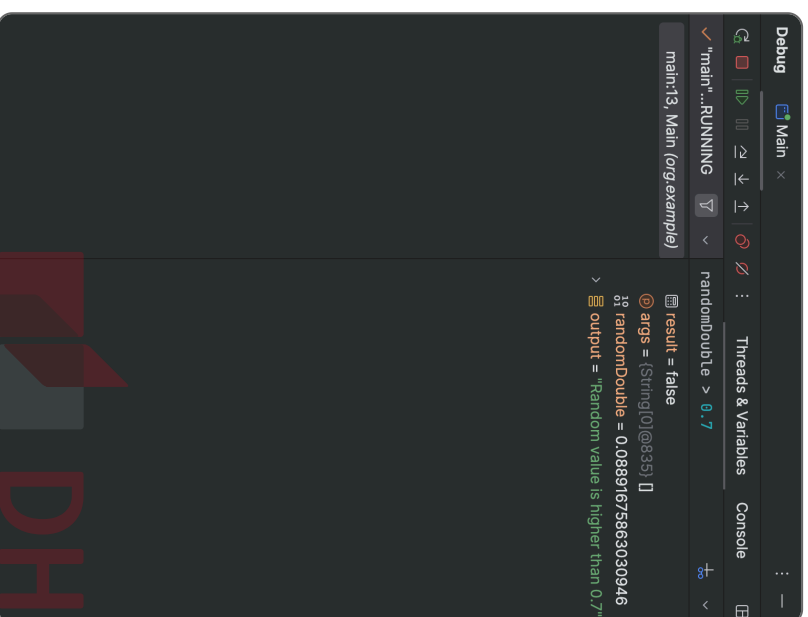
VS CODE



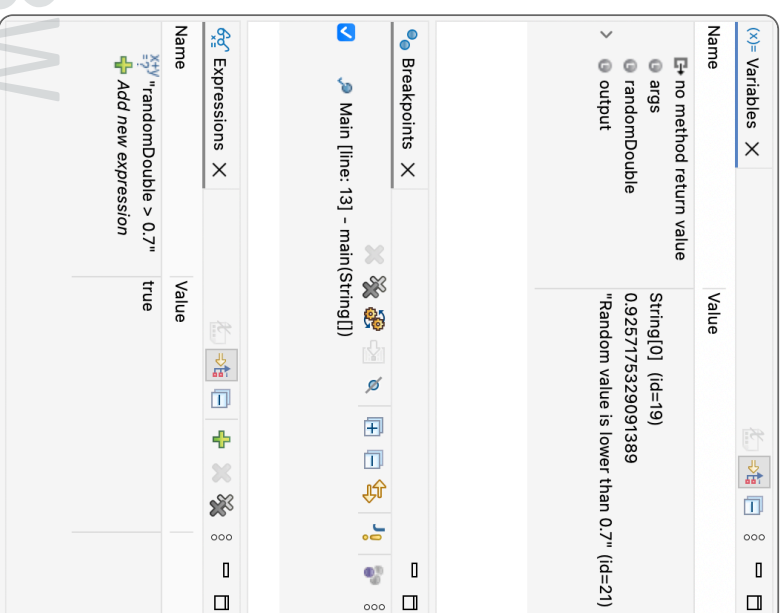
- 👉 **Step Into:** In die Methode hineinspringen
- 👉 **Step Over:** Die Methode ausführen, ohne hineinzuspringen
- 👉 **Step Out:** Aus der aktuellen Methode herausspringen
- 👉 **Resume:** Programm bis zum nächsten Breakpoint fortsetzen
- 👉 **Evaluate Expression:** Beliebigen Ausdruck zur Laufzeit auswerten
- 👉 **Run to Cursor:** Programm bis zur Cursor-Position ausführen

DEBUGGER IN DER IDE

INTELLIJ IDEA



ECLIPSE IDE



VS CODE



BREAKPOINTS

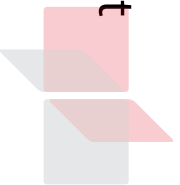


Anhalten bei Breakpoints nicht effizient bei großen Softwareprojekten



Besser: Gezieltes Überwachen/Anhalten des Programms notwendig

Breakpoint-Art	Beschreibung
Standard Breakpoint	Hält die Programmausführung an einer bestimmten Codezeile an.
Conditional Breakpoint	Hält die Ausführung nur an, wenn eine bestimmte Bedingung erfüllt ist.
Logpoint	Schreibt eine Nachricht in die Konsole, ohne die Ausführung zu unterbrechen.
Exception Breakpoint	Hält die Ausführung an, wenn eine bestimmte Exception geworfen wird.



Duale Hochschule
Baden-Württemberg

WEITERE MÖGLICHKEITEN IM DEBUGGER



ERWEITERTE FEATURES



Ändern von Variablenwerten: Zur Laufzeit Werte modifizieren und Auswirkungen testen



Manuelles Ausführen von Methoden: Beliebige Methoden während des Anhaltens aufrufen



Hot Swapping: Code-Änderungen ohne Neustart anwenden



REMOTE DEBUGGING



Server-Debugging: Code auf entfernten Servern debuggen



Debug-Port: Verbindung über spezielle Debug-Ports (z.B. Port 5005)



Mobile Apps: Debugging von Apps auf Geräten oder Emulatoren



Wichtiger Hinweis: Remote Debugging in Produktionsumgebungen sollte nur in Ausnahmefällen und mit entsprechenden Sicherheitsvorkehrungen durchgeführt werden.



Duale Hochschule
Baden-Württemberg